



Profiling CUDA Kernels with NVIDIA NSight Compute

- [Profiling CUDA Kernels with NVIDIA NSight Compute](#)
- [Setting up your working environment](#)
 - [Local Installation](#)
 - [Running on CHTC](#)
 - [Running on CHTC](#)
- [\(A brief\) Introduction to GPU Programming](#)
 - [The CUDA Programming Model](#)
- [NVIDIA NSight Compute](#)
 - [Getting Started](#)
 - [Using the Nsight Compute CLI](#)
 - [Using the Nsight Compute GUI](#)
 - [Fixing the Code](#)
- [Profiling MadGraph_aMC@NLO](#)
 - [Profiling MadGraph Kernels](#)
 - [Analysis with NSight Compute](#)
 - [Further Reading](#)
- [Appendix](#)

Setting up your working environment

Local Installation

Since we're using GPUs on CHTC nodes, no software installation on your own device is required. (n.b. This is not entirely true — NSight Compute, the profiling tool we're going to use has a GUI that cannot be used on HPC systems directly. One has to either download and analyze the profiling report or SSH into an interactive node using NSight on their local machine.)

At UW, we currently have two options to run large-scale GPU jobs. We can either run them on a node on the CHTC cluster, or one of our local CMS machines with a GPU.

Running on CHTC

All jobs submitted here run on a single shared node with a 40GB NVIDIA A100 GPU. The CUDA compiler `nvcc` currently runs out of a singularity image that must be run on the node before your jobs can execute. If the singularity container does not already exist, it can be recreated by following the steps below. The singularity definition file can be found in the [appendix](#)

```
# This command will build a singularity image from the definition file
# {singularity_def_file}.def
sudo singularity build {image_name}.sif {singularity_def_file}.def

# Once the image is built, it can be deployed in a standalone environment on any
# system with singularity as below:
singularity run {image_name}.sif
```

Once your singularity image is ready, we can now run a job. CHTC manages jobs using the HTCondor framework. To submit a job to the requested node, we run the following batch script:

```
universe = vanilla
output = nsight-compute/log/ncu-$(CLUSTER).$(PROCESS).out
error = nsight-compute/log/ncu-$(CLUSTER).$(PROCESS).err
log = nsight-compute/log/ncu-$(CLUSTER).$(PROCESS).log

transfer_executable = True
executable = {profiling-script}.sh
should_transfer_files = yes
transfer_input_files = {image_name}.sif
transfer_output_files = {gpu-profiling-report}
when_to_transfer_output = ON_EXIT

+WantGPULab = true
+InteractiveJob = true
+GPUJobLength = "short"

request_gpus = 1
request_cpus = 1
request_memory = 2GB
request_disk = 7GB

+SingularityImage = "{image_name}.sif"
Requirements = HasSingularity
Requirements = GPUs_DeviceName == "NVIDIA A100-SXM4-40GB"
requirements = Machine == "gpulab2004.chtc.wisc.edu" && OpSysMajorVer == 8

queue 1
```

Information on managing a condor job can be found on the [HTCondor Documentation Page](#).

Running on CHTC

At present, it is not possible to run profiling jobs on `cmsgpu01` without sudo access.

(A brief) Introduction to GPU Programming

This section assumes familiarity with the CUDA programming model and fundamentals of GPU hardware. It is also assumed that the reader is comfortable with programming in C/C++. For a more comprehensive introduction to CUDA, check out [CUDA Training Series by Oakland National Laboratory](#).

The CUDA Programming Model

In order to perform computation, the host launches a kernel that is executed simultaneously by many threads on the device. It is important to note that a GPU thread differs from a regular CPU thread. The launch configuration defines the number of thread blocks and the number of threads in each block that execute the kernel. Each thread is associated with an intrinsic index, which can be used to calculate and access memory locations. Each thread has its own context and set of private variables. All threads have access to the GPU global memory, but there is no general way to synchronize execution of arbitrary threads.

At the hardware level, the CUDA programming model groups 32 threads in groups called warps. All members of a warp execute the same instruction in lockstep. If threads within a same warp take two different execution paths on

encountering a branching condition, both branches are executed sequentially with non-participating threads in that branch being inactive. This condition is called branch divergence, and is a major source of latency.

Memory access to the GPU memory occur in blocks of specific sizes (32B, 64B, 128B, etc.). To efficiently access memory with minimum latency, all threads in a warp need to access elements of data that **lie in the same cache line**. This is called *coalesced memory access*.

Threads are further grouped into thread blocks, which reside on a piece of hardware called the Streaming Multiprocessor (SM). Each block is assigned to a single SM. A SM consists of many computing elements, schedulers, and an extremely fast on-chip memory called shared memory. It is possible to synchronize threads at a block level. For optimal performance, blocks should have more than one warp. If sufficient resources are available, it is possible for an SM to have multiple active thread blocks simultaneously; however, these blocks cannot share data with each other.

In order to hide latencies it is recommended to “over-subscribe” the GPU. There should be many more blocks than SMs present on the device. Also in order to ensure a good occupancy of the CUDA cores there should be more warps active on a given SM than SIMT units. This way while some warps of threads are idle waiting for some memory operations to complete, others use the CUDA cores, thus ensuring a high occupancy of the GPU.

NVIDIA NSight Compute

Nsight Compute is a powerful profiling tool that provides detailed performance data and offers guidance for optimizing CUDA kernels. Nsight Compute offers both a GUI as well as a command line interface (CLI), and provides full support for CUDA 10.0+ and Volta+ GPUs.

Getting Started

As a pedagogical exercise in learning how to use Nsight Compute, we’re going to profile a CUDA kernel that does a matrix-matrix element-wise add operation using a 2D CUDA grid configuration.

```
#include <iostream>

const size_t size_w = 1024;
const size_t size_h = 1024;

typedef unsigned mytype;
typedef mytype arr_t[size_w];
const mytype A_val = 1;
const mytype B_val = 2;

__global__ void matrix_add_2D(const arr_t* __restrict__ A, const arr_t* __restrict__ B, arr_t* __restrict__ C)
{
    size_t idx = threadIdx.x + blockDim.x * (size_t) blockIdx.x;
    size_t idy = threadIdx.y + blockDim.y * (size_t) blockIdx.y;
    if((idx < sh) && (idy < sw))
        C[idx][idy] = A[idx][idy] + B[idx][idy];
}

int main()
{
    arr_t *A, *B, *C;
    size_t ds = size_w * size_h * sizeof(mytype);
    cudaError_t err = cudaMallocManaged(&A, ds);
    if (err != cudaSuccess)
    {
        std::cout << "CUDA error: " << cudaGetErrorString(err) << std::endl;
    }
}
```

```

        return 0;
    }
    cudaMallocManaged(&B, ds);
    cudaMallocManaged(&C, ds);
    for (int x = 0; x < size_h; x++)
    {
        for (int y = 0; y < size_h; y++)
        {
            A[x][y] = A_val;
            B[x][y] = B_val;
            C[x][y] = 0;
        }
    }

    int attr = 0;
    cudaDeviceGetAttribute(&attr, cudaDevAttrConcurrentManagedAccess, 0);
    if (attr)
    {
        cudaMemPrefetchAsync(A, ds, 0);
        cudaMemPrefetchAsync(B, ds, 0);
        cudaMemPrefetchAsync(C, ds, 0);
    }
    dim3 threads(32, 32);
    dim3 blocks((size_w + threads.x - 1)/threads.x, (size_h + threads.y - 1)/threads.y);
    matrix_add_2D<<<blocks, threads>>>(A, B, C, size_w, size_h);
    cudaDeviceSynchronize();
    err = cudaGetLastError();
    if (err != cudaSuccess)
    {
        std::cout << "CUDA error: " << cudaGetErrorString(err) << std::endl;
        return 0;
    }

    for (int x = 0; x < size_h; x++)
    {
        for(int y = 0; y < size_w; y++)
        {
            if (C[x][y] != A_val + B_val)
            {
                std::cout << "mismatch at: (" << x << ", " << y << ") was: " << C[x][y] << " should be: " << A_val + B_val << std::endl;
                return 0;
            }
        }
    }
    std::cout << "Success!" << std::endl;
    return 0;
}

```

The code should be fairly straightforward to understand, but there are two main features:

1. **Managed memory:** Data is allocated using managed allocations. For GPUs that support on-demand paging, data is prefetched to avoid performance overheads on the kernel.

2. **2D:** We are launching a 2D grid of blocks, along with a 2D threadblock shape. This allows us to use simple 2D arrays, without requiring arrays of pointers.

As a CUDA programmer, the two most important optimization priorities are one, to expose enough parallel work to the GPU to avoid any idle cycles, and two, making efficient use of the memory subsystem. In this code, we do not have enough data to meet the first objective. We only use global memory, so we’re interested in efficient use of global memory via coalesced memory accesses.

Using the Nsight Compute CLI

For most purposes, we can get similar data using the CLI or the GUI, but the CLI is easier to use if you know specifically what data you are looking for, and/or if you want to use command line automation.

If the path to the profiler tool is not setup by default, you want to add `/usr/local/cuda/bin/` to your `PATH` variable. You can verify this by running `ncu --version`. Before running the profiler, you must first compile your CUDA code with `nvcc` compiler. To do so, run `bash nvcc -o example.o example.cu --lineinfo`

Please note that Nsight Compute is not supported on devices with CC 6.0 and lower.

The `--lineinfo` flag allows Nsight to cross-reference written code with CUDA opcode and suggest targeted optimization strategies by isolating bottlenecks and inefficiencies in user code. For more compilation options, refer to the [nvcc documentation](#).

Now, we can run the profiler on the compiled object file.

```
$ ncu ./example
==PROF== Connected to process 3896054 (/afs/hep.wisc.edu/user/<user>/public/gpu-profiling/example.o)
==PROF== Profiling "matrix_add_2D" - 0: 0%...50%...100% - 10 passes
Success!
==PROF== Disconnected from process 3896054
[3896054] example.o@127.0.0.1
matrix_add_2D(const unsigned int (*)[1024], const unsigned int (*)[1024], unsigned int (*)[1024], unsigned int (*)[1024])
Section: GPU Speed Of Light Throughput
-----
Metric Name                Metric Unit Metric Value
-----
DRAM Frequency             Ghz          4.92
SM Frequency                Mhz          584.90
Elapsed Cycles              cycle        186,106
Memory Throughput           %            43.35
DRAM Throughput             %            32.94
Duration                   us           318.18
L1/TEX Cache Throughput     %            86.70
L2 Cache Throughput         %            27.16
SM Active Cycles            cycle        177,709.58
Compute (SM) Throughput     %            4.42
-----

OPT This kernel exhibits low compute throughput and memory bandwidth utilization relative to the peak of this device. Achieved compute throughput and/or memory bandwidth below 60.0% of peak typical values. Look at Scheduler Statistics and Warp State Statistics for potential reasons.

Section: Launch Statistics
-----
Metric Name                Metric Unit Metric Value
-----
```

Block Size		1,024
Function Cache Configuration	CachePreferNone	
Grid Size		1,024
Registers Per Thread	register/thread	16
Shared Memory Configuration Size	Kbyte	32.77
Driver Shared Memory Per Block	byte/block	0
Dynamic Shared Memory Per Block	byte/block	0
Static Shared Memory Per Block	byte/block	0
# SMs	SM	40
Threads	thread	1,048,576
Uses Green Context		0
Waves Per SM		25.60

Section: Occupancy

Metric Name	Metric Unit	Metric Value
Block Limit SM	block	16
Block Limit Registers	block	4
Block Limit Shared Mem	block	16
Block Limit Warps	block	1
Theoretical Active Warps per SM	warp	32
Theoretical Occupancy	%	100
Achieved Occupancy	%	73.27
Achieved Active Warps Per SM	warp	23.45

OPT Est. Local Speedup: 26.73%

The difference between calculated theoretical (100.0%) and measured achieved occupancy (73.3%) result of warp scheduling overheads or workload imbalances during the kernel execution. Load imbalance can occur between warps within a block as well as across blocks of the same kernel. See the CUDA Performance Guide (<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#occupancy>) for more information on optimizing occupancy.

Section: GPU and Memory Workload Distribution

Metric Name	Metric Unit	Metric Value
Average DRAM Active Cycles	cycle	515,437.50
Total DRAM Elapsed Cycles	cycle	12,516,352
Average L1 Active Cycles	cycle	177,709.58
Total L1 Elapsed Cycles	cycle	7,410,016
Average L2 Active Cycles	cycle	266,296.69
Total L2 Elapsed Cycles	cycle	8,703,264
Average SM Active Cycles	cycle	177,709.58
Total SM Elapsed Cycles	cycle	7,410,016
Average SMSP Active Cycles	cycle	170,211.94
Total SMSP Elapsed Cycles	cycle	29,640,064

That's a lot of output! If your code has multiple kernel invocations, details page data will be gathered and displayed for each. We won't try and go through it all in detail, but notice there are major sections for SOL (speed of light — comparisons against the best possible behaviour supported on the device), compute analysis, memory analysis, scheduler, warp state, instruction, launch statistics, and occupancy analysis. Also notice section-specific optimization strategies are that suggested by Nsight. While these suggestions might not always be helpful, there are an excellent starting point.

You can optionally select which of these sections are collected and displayed with command-line flags and parameters. The list of all possible command-line flags can be found in the [NSight CLI documentation](#).

In our example, we're interested in studying global load/store transaction. For global load transactions, we will use `lltex__t_sectors_pipe_lsu_mem_global_op_ld.sum` and for global load requests we will use `lltex__t_requests_pipe_lsu_mem_global_op_ld.sum`. At this point you might be wondering about the length of these metric names and naming convention. There is a method to the naming, and you can review it in the documentation. The naming convention is intended to make it easier to understand what a metric represents from its name. Briefly, the metric name preceding the period identifies where in the architecture the data is being collected, and the token after the period identifies mathematically how the number is gathered. For most base metric names on Volta and newer, suffixes (like `.sum`, `.avg`, ...) exist that together with the base name make up the actual metric name that can be collected. Once you understand this concept for one metric, you can easily apply it to almost any other available metric on this architecture.

We gather these metrics by passing their names as a command line argument

```
$ ncu --metrics lltex__t_sectors_pipe_lsu_mem_global_op_ld.sum,lttex__t_requests_pipe_lsu_mem_global_op
mple.o
==PROF== Connected to process 3906982 (/afs/hep.wisc.edu/user/athete/public/gpu-profiling/example.o)
==PROF== Profiling "matrix_add_2D" - 0: 0%...50%...100% - 4 passes
Success!
==PROF== Disconnected from process 3906982
[3906982] example.o@127.0.0.1
matrix_add_2D(const unsigned int (*)[1024], const unsigned int (*)[1024], unsigned int (*)[1024], unsi
Section: Command line profiler metrics
```

Metric Name	Metric Unit	Metric Value
lltex__t_requests_pipe_lsu_mem_global_op_ld.sum		65,536
lltex__t_sectors_pipe_lsu_mem_global_op_ld.sum	sector	2,097,152

This first metric above represents the denominator (requests) of the desired measurement (transactions per request) and the second metric represents the numerator (transactions). If we divide these, we get 32 transactions per request. Therefore, each thread in the warp is generating a separate transaction. This is a good indication that our access pattern (reading, in this case) is not coalesced.

Using the Nsight Compute GUI

If instead, we are interested in gathering and analyzing these metrics using the GUI, we need an active X11 session to run the GUI app version in.

N.B. Ideally, once an active X11 session is established, you type `ncu-ui` to open the GUI and everything should work, but unfortunately I have not been able to get it running by following these steps. Instead, you have to export the profiling report as a `ncu-rep` file, download it to your local machine, and open it there.

Fixing the Code

The reason for high number of transactions per request in this code is due to how elements are being indexed.

```
| ... C[idx][idy] = A[idx][idy] + B[idx][idy]
```

The index built with `threadIdx.x` (i.e. `idx`) should appear in the last subscript for coalesced access across a warp; instead it appears in the first subscript. While either method can give correct results, they are not the same from a performance perspective. This arrangement results in each thread in a warp accessing data in a “column” in memory, rather than a “row” (i.e. adjacent). We can fix this by modifying our kernel code as follows:

```
__global__ void matrix_add_2D(const arr_t* __restrict__ A, const arr_t* __restrict__ B, arr_t* __restrict__ C)
{
    size_t idx = threadIdx.x + blockDim.x * (size_t) blockIdx.x;
    size_t idy = threadIdx.y + blockDim.y * (size_t) blockIdx.y;
    if((idx < sh) && (idy < sw))
        C[idy][idx] = A[idy][idx] + B[idy][idx];
}
```

Recompiling and profiling again, we see:

```
$ ncu --metrics lltex__t_sectors_pipe_lsu_mem_global_op_ld.sum,lltex__t_requests_pipe_lsu_mem_global_op_ld.sum
==PROF== Connected to process 4192288 (/afs/hep.wisc.edu/user/athete/public/gpu-profiling/example.o)
==PROF== Profiling "matrix_add_2D" - 0: 0%...50%...100% - 4 passes
Success!
==PROF== Disconnected from process 4192288
[4192288] example.o@127.0.0.1
matrix_add_2D(const unsigned int (*)[1024], const unsigned int (*)[1024], unsigned int (*)[1024], unsigned int (*)[1024])
Section: Command line profiler metrics
```

Metric Name	Metric Unit	Metric Value
lltex__t_requests_pipe_lsu_mem_global_op_ld.sum		65,536
lltex__t_sectors_pipe_lsu_mem_global_op_ld.sum	sector	262,144

Now the ratio of the metrics is 4:1 (transactions per request), indicating the desired transaction size of 32 bytes is achieved, and the efficiency of loads (and stores) is substantially improved over the previous case.

Profiling MadGraph_aMC@NLO

MadGraph_aMC@NLO (MG5aMC) is a code generator, written in Python, which allows the generation of the code for a chosen physics process in many languages. There is an ongoing effort to reengineer the Madgraph5 event generator, and port the matrix element calculation from the current Fortran version to faster implementations using C++ based programming models on CPUs and GPUs. For a complete overview of the [MadGraph4GPU project](#) and its [codebase](#), see the highlighted links.

Profiling MadGraph Kernels

We can use the NVIDIA NSight software suite to profile GPU kernels in the CUDA version. First, we must clone the code repository, and build a CUDA-enabled standalone MadGraph application to profile. The profiling is performed on a `g g > t t g g` process.

Create a shell script that builds the MadGraph application and runs the profiler. This shell script is submitted to a HTCondor GPU node and run inside the singularity container we built earlier. An example of a such a script is given below.

```
origdir=$(pwd)
git clone https://github.com/madgraph5/madgraph4gpu.git
cd madgraph4gpu
export MADGRAPH4GPU_HOME=`pwd`
```



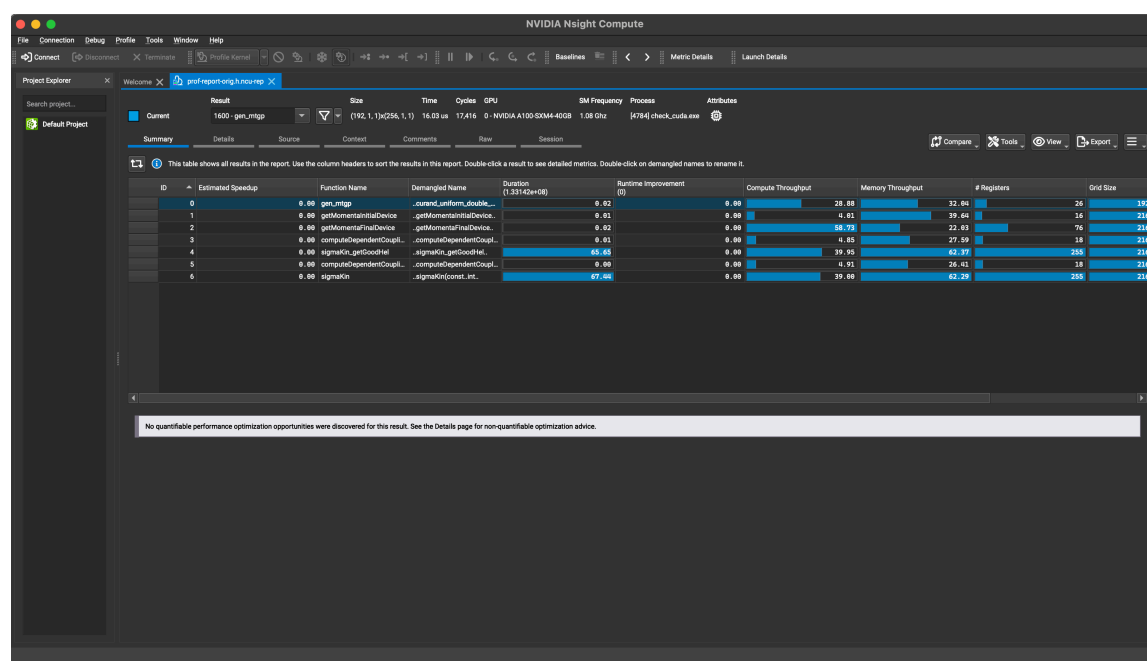
```
cd epochX/cudacpp/gg_ttgg.sa/SubProcesses/P1_Sigma_sm_gg_ttxgg
sed -i 's/###CUFLAGS+= --maxrregcount 128/CUFLAGS+= --maxrregcount 255/' makefile
make cleanall
# Builds standalone MG application
make BACKEND=cuda FPTYPE=d HELINL=0
# Run profiler on standalone app
# Args: {num_GPU_threads} {num_GPU_regs_per_thread} {num_iterations}
ncu --set full -o $origdir/report ./check_cuda.exe -p 216 256 1
```

Then, we can use the job submission script shown **before** to submit a profiling job.

If the job runs successfully, you should see three log files in your working directory in addition to a `.ncu-rep` report file that is tranfered from the HTCondor node after the job ends. Download the report file to your local computer, or another machine with a working Nsight Compute GUI for analy

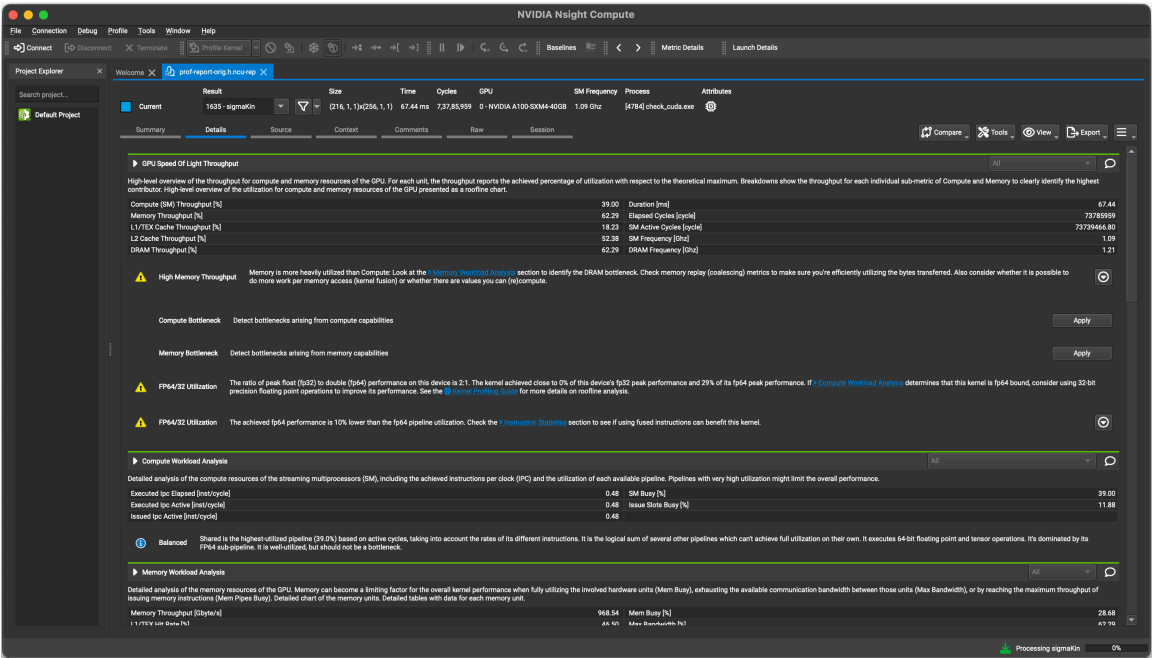
Analysis with NSight Compute

On opening the report, you should see a screen that looks something like this:



This screen lists all the kernels that were profiled by the application, along with a couple of superficial metrics like duration, compute throughput, memory throughput, number of registers used per thread, launch configuration, and so on. To see more kernel-specific details, click on the row with that kernel.

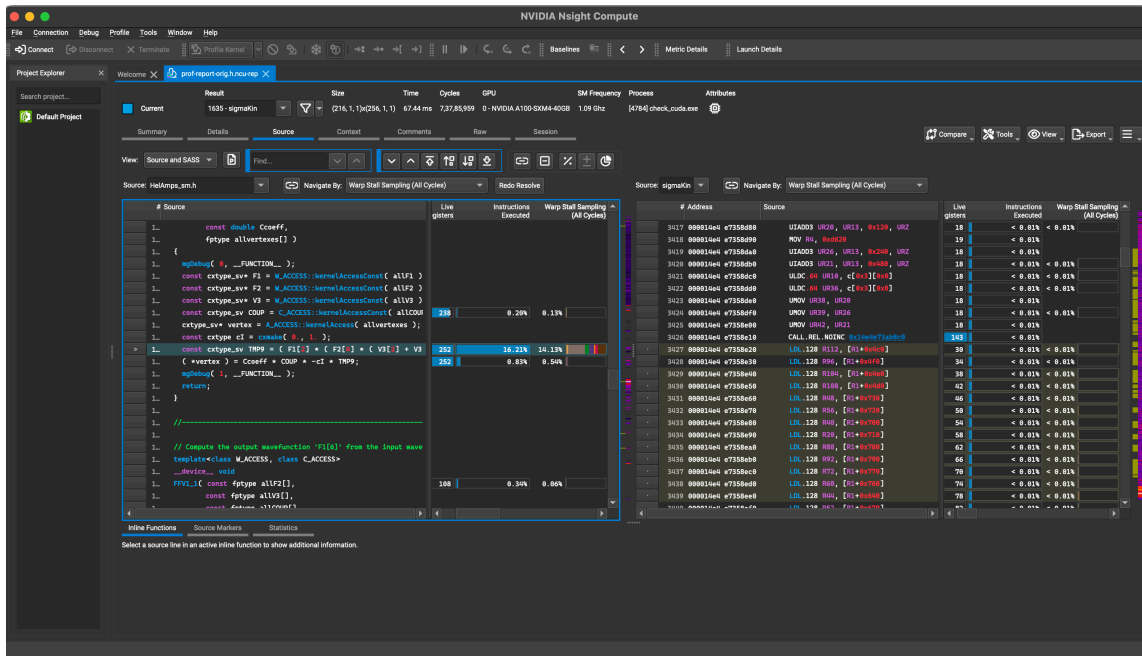
A much more detailed report screen should appear with multiple sections.



It is useful to begin by analyzing the data in the GPU Speed on Light section. This tab provides a high-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. A Roofline plot for the kernel should also appear, which indicates how efficiently single and double precision operations are performed by the GPU.

At the end of each section, warnings and profiling advice indicate potential performance optimization you can perform to improve kernel behaviour across a wide range of GPU units. For example, in the kernel chosen for this example, memory is being more heavily utilized than compute, which is indicative of poor memory accesses or insufficient utilization of compute units within the GPU. The compute and memory workload analysis tabs are helpful to pinpoint the exact cause for this warning. The memory workload analysis tab also includes a schematic that shows the various logical and physical memory pipeline accesses that the kernel performs over its lifetime.

Scheduler and Warp Stall tabs can be used to determine if we are using the appropriate kernel launch configuration that prioritizes efficient memory and compute utilization. Sometimes, memory and compute bottlenecks in the code can cause the GPU to sit idle for longer than needed. The source section can identify the exact line of SASS as well as the kernel source code that is the cause of the bottleneck. Either click on the Source tab, or navigate to the Source Counters section in the main profiling report to identify the top five warp stall locations in the source code. To obtain meaningful data from this section, compile the kernel with the `-lineinfo` option.



Further Reading

This is by no means an exhaustive guide to learning to use NSight or profiling tools. It is impossible for a single guide to cover all possible results you will encounter or edge cases that you might have to work around. Profiling is incredibly “hacky” and requires a lot of time and patience to understand the inner workings of the GPU and the code to meaningfully interpret the data and suggestions provided by the profiler. Listed below are some more resources that are a good starting point once you’re more comfortable with CUDA and GPU hardware.

- Official NVIDIA Profiling Guide
- A Hands-On NSight training exercise

Appendix

Paste the contents below into a file with the extension `.sif`. This creates a singularity definition file that you can use to build a singularity image to run GPU jobs on the CHTC cluster.

```
Bootstrap: docker
From: centos:7

%post
# Install required dependencies
yum -y update
yum -y install wget sudo cmake which git
yum -y install centos-release-scl

# Install GCC 10.2 and related packages
yum -y install devtoolset-10-gcc devtoolset-10-gcc-c++ devtoolset-10-gcc-gfortran

# TODO: Update Drivers and CUDA links
# Download and install NVIDIA drivers
wget https://us.download.nvidia.com/XFree86/Linux-x86_64/525.116.04/NVIDIA-Linux-x86_64-525.116.04.run
chmod +x NVIDIA-Linux-x86_64-525.116.04.run
./NVIDIA-Linux-x86_64-525.116.04.run -s --no-kernel-module
```

```
# Download and install CUDA 12.1
wget https://developer.download.nvidia.com/compute/cuda/12.1.1/local_installers/cuda_12.1.1_530.30.02_linux
chmod +x cuda_12.1.1_530.30.02_linux.run
./cuda_12.1.1_530.30.02_linux.run --silent --toolkit --override

# Clean up
rm NVIDIA-Linux-x86_64-525.116.04.run cuda_12.1.1_530.30.02_linux.run

%environment
# Set environment variables for NVIDIA drivers and CUDA

scl enable devtoolset-10 'echo "GCC 10.2 enabled"'
export PATH="/opt/rh/devtoolset-10/root/bin:${PATH}"
export PATH="/usr/local/nvidia/bin:${PATH}"
export LD_LIBRARY_PATH="/usr/local/nvidia/lib64:${LD_LIBRARY_PATH}"
export PATH="/usr/local/cuda-12.1/bin:${PATH}"
export LD_LIBRARY_PATH="/usr/local/cuda-12.1/lib64:${LD_LIBRARY_PATH}"
export FC=$(which gfortran)
export CC=$(which gcc)
export CXX=$(which g++)
```

Department of Physics, Chamberlin Hall, 1150 University Avenue, Madison, Wisconsin 53706

Contact us at: help@hep.wisc.edu | tel: 608-262-2281

© 2011 Board of Regents of the [University of Wisconsin System](#)